Phase 2 Machine Learning Guide

Theory and examples for boosting, evaluation, tuning, SVM, Naive Bayes, and feature engineering.

Learning Order

1. XGBoost and LightGBM
2. Cross validation
3. Model evaluation metrics
4. Hyperparameter tuning
5. SVM
6. Naive Bayes
7. Feature engineering
8. Mini project workflow

1. XGBoost and LightGBM

Theory

XGBoost means Extreme Gradient Boosting. It is a powerful tree-based machine learning algorithm.

Random Forest builds many trees independently and combines their results. XGBoost builds trees one by one. Each new tree tries to correct the mistakes made by previous trees.

Simple idea:

```
Tree 1 makes predictions
Tree 2 fixes errors from Tree 1
Tree 3 fixes remaining errors
Final prediction = combined result of all trees
```

This technique is called boosting.

Random Forest vs XGBoost

```
Random Forest:
Trees are trained mostly independently.
Final result is voting or averaging.

XGBoost:
Trees are trained sequentially.
Each new tree learns from previous errors.
```

Why XGBoost is useful

- Very strong for tabular data
- Works for classification and regression
- Can learn nonlinear patterns
- Includes regularization to reduce overfitting
- Often performs better than basic models

Important XGBoost hyperparameters

```
n_estimators       Number of boosting trees
learning_rate      How fast the model learns
max_depth          Maximum depth of each tree
subsample          Fraction of rows used per tree
colsample_bytree   Fraction of columns used per tree
gamma              Minimum loss reduction needed to split
reg_alpha          L1 regularization
reg_lambda         L2 regularization
```

XGBoost classification example

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from xgboost import XGBClassifier

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
    eval_metric="logloss"
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

XGBoost regression example

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor

data = fetch_california_housing()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

LightGBM

LightGBM is another gradient boosting library. It is often faster than XGBoost on large datasets.

Use LightGBM when:

- The dataset is large
- You need fast training
- You have many rows or many features

Basic LightGBM example:

```
from lightgbm import LGBMClassifier

model = LGBMClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=-1,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

2. Cross Validation

Theory

A single train-test split can be lucky or unlucky. Cross validation gives a more reliable performance estimate by testing the model on multiple splits.

In 5-fold cross validation:

```
Dataset is split into 5 parts.
Round 1: fold 1 is validation, folds 2-5 are training.
Round 2: fold 2 is validation, folds 1,3,4,5 are training.
This repeats until every fold has been validation once.
Final score = average of all validation scores.
```

Why use cross validation

- More reliable than one split
- Helps compare models fairly
- Reduces dependency on one random split

Cross validation example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

data = load_iris()
X = data.data
y = data.target

model = RandomForestClassifier(random_state=42)

scores = cross_val_score(
    model,
    X,
    y,
    cv=5,
    scoring="accuracy"
)

print("Scores:", scores)
print("Average accuracy:", scores.mean())
```

Stratified cross validation

For classification, StratifiedKFold keeps class ratios similar in every fold. This is useful when classes are imbalanced.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
model = RandomForestClassifier(random_state=42)

scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")

print(scores)
print(scores.mean())
```

Important warning: data leakage

Do not scale or impute using the full dataset before cross validation. Put preprocessing inside a Pipeline.

Wrong:

```
X_scaled = scaler.fit_transform(X)
cross_val_score(model, X_scaled, y, cv=5)
```

Correct:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", SVC())
])

scores = cross_val_score(pipeline, X, y, cv=5)
```

3. Model Evaluation Metrics

Theory

Accuracy is not always enough. If the dataset is imbalanced, a model can have high accuracy but still fail on the important class.

Example:

```
100 patients
95 healthy
5 sick

If model predicts everyone is healthy:
Accuracy = 95%
But it finds 0 sick patients.
```

Classification metrics

```
Accuracy   Correct predictions / all predictions
Precision  Of predicted positives, how many were truly positive
Recall     Of actual positives, how many were found
F1-score   Balance of precision and recall
ROC-AUC    Ranking quality for binary classification
```

Confusion matrix

```
                Predicted 0   Predicted 1
Actual 0        TN            FP
Actual 1        FN            TP

TN = true negative
FP = false positive
FN = false negative
TP = true positive
```

Classification example

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Regression metrics

```
MAE     Mean absolute error
MSE     Mean squared error
RMSE    Square root of MSE
R2      How much variance the model explains
```

Regression example:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
```

Which metric should you choose?

```
Balanced classification problem: accuracy can be OK.
Imbalanced classification problem: use precision, recall, F1, ROC-AUC.
Medical detection: recall is often very important.
Spam detection: precision can be important.
Regression: use MAE, RMSE, and R2.
```

4. Hyperparameter Tuning

Theory

Parameters are learned by the model during training. Hyperparameters are settings chosen before training.

Example:

```
RandomForestClassifier(
    n_estimators=100,
    max_depth=5
)
```

Here, ‘n_estimators‘ and ‘max_depth‘ are hyperparameters.

Why tune hyperparameters?

Default settings can work, but they are not always best. Tuning tries different settings to find a better model.

GridSearchCV

GridSearchCV does:

```
Grid Search + Cross Validation
```

It tries every hyperparameter combination you provide, then uses cross validation to score each combination.

Example:

```
n_estimators = [50, 100, 200]
max_depth = [3, 5, 10]

Total combinations = 3 x 3 = 9
If cv=5, model trains 9 x 5 = 45 times.
```

GridSearchCV example

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 10, None],
    "min_samples_split": [2, 5, 10]
}

grid = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score:", grid.best_score_)

best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

print("Test accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

RandomizedSearchCV

RandomizedSearchCV samples random combinations instead of trying every combination. It is useful for large search spaces.

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier

param_dist = {
    "n_estimators": [50, 100, 200, 300, 500],
    "max_depth": [3, 5, 10, 15, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"]
}

search = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring="accuracy",
    n_jobs=-1,
    random_state=42
)

search.fit(X_train, y_train)

print(search.best_params_)
print(search.best_score_)
```

XGBoost tuning example

```
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier

model = XGBClassifier(
    random_state=42,
    eval_metric="logloss"
)

param_grid = {
    "n_estimators": [50, 100, 200],
    "learning_rate": [0.01, 0.1, 0.2],
    "max_depth": [3, 5, 7],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0]
}

grid = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score:", grid.best_score_)
```

Important rule

Use training data for tuning. Use test data only once at the end for final evaluation.

5. SVM

Theory

SVM means Support Vector Machine. It is commonly used for classification.

SVM finds the best boundary between classes. The best boundary is the one with the largest margin from the closest points of each class.

```
Class A         margin      boundary      margin         Class B
A A A                         |
A A                           |
A                             |                        B B B
```

The closest points to the boundary are called support vectors.

Kernel trick

If a straight line cannot separate classes, SVM can use a kernel to make a nonlinear boundary.

Common kernels:

```
linear    Straight boundary
rbf       Curved boundary, most common
poly      Polynomial boundary
sigmoid   Neural-network-like boundary
```

Important SVM hyperparameters

```
C       Controls how strict the model is about mistakes.
gamma   Controls influence distance for rbf/poly/sigmoid kernels.
kernel  Controls boundary shape.
```

Small C allows more mistakes and creates a smoother boundary. Large C tries to avoid mistakes but can overfit.

Small gamma creates smoother boundaries. Large gamma creates complex boundaries and can overfit.

Important: scale features

SVM is sensitive to feature scale. Always scale numerical features before SVM.

SVM example with Pipeline

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", C=1.0, gamma="scale"))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

SVM with GridSearchCV

```
from sklearn.model_selection import GridSearchCV

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC())
])

param_grid = {
    "svm__kernel": ["linear", "rbf"],
    "svm__C": [0.1, 1, 10],
    "svm__gamma": ["scale", 0.01, 0.1, 1]
}

grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score:", grid.best_score_)
```

When to use SVM

Use SVM when:

- Dataset is small or medium
- Features are mostly numerical
- You can scale the data
- You need strong classification

Avoid SVM when:

- Dataset is very large
- Training speed is very important
- Interpretability is very important

6. Naive Bayes

Theory

Naive Bayes is a probability-based classifier. It predicts the class with the highest probability.

Example:

```
Email text: "win free money"

P(spam | text) = 0.92
P(not spam | text) = 0.08

Prediction = spam
```

It is called naive because it assumes features are independent. This assumption is simple, but the model still works well for many text problems.

Bayes theorem

```
P(Class | Data) = P(Data | Class) * P(Class) / P(Data)
```

Meaning:

```
Probability of class after seeing the data
```

Types of Naive Bayes

```
GaussianNB      Continuous numerical features
MultinomialNB   Count features, useful for text
BernoulliNB     Binary features, such as word present/absent
```

Gaussian Naive Bayes example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Text classification example

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

texts = [
    "win money now",
    "free prize winner",
    "hello how are you",
    "meeting tomorrow"
]

labels = [
    "spam",
    "spam",
    "not spam",
    "not spam"
]

model = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("nb", MultinomialNB())
])

model.fit(texts, labels)

prediction = model.predict(["win free money"])
print(prediction)
```

Advantages

- Very fast
- Simple
- Works well for text classification
- Good baseline model
- Works with small datasets

Disadvantages

- Assumes features are independent
- May miss complex relationships
- Often less powerful than boosted trees for tabular data

7. Feature Engineering

Theory

Feature engineering means creating, changing, selecting, or transforming input columns to make data more useful for machine learning.

Good features can improve model performance more than changing algorithms.

Common techniques

```
Handle missing values
Encode categorical columns
Scale numerical columns
Create new features
Extract date features
Remove useless columns
```

Handling missing values

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="mean")
X_filled = imputer.fit_transform(X)
```

Common strategies:

```
mean
median
most_frequent
constant
```

Encoding categorical data

Machine learning models need numbers, not text.

One-hot encoding example:

```
City
Delhi
Mumbai
Chennai

Becomes:

City_Delhi  City_Mumbai  City_Chennai
1           0            0
0           1            0
0           0            1
```

Python:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
encoded = encoder.fit_transform(X[["City"]])
```

Scaling numerical data

Scaling is important for SVM, KNN, logistic regression, linear regression, and neural networks.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Creating new features

Example: create BMI from height and weight.

```
df["BMI"] = df["Weight"] / (df["Height"] ** 2)
```

Date features

```
df["date"] = pd.to_datetime(df["date"])
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["day_of_week"] = df["date"].dt.dayofweek
df["is_weekend"] = df["day_of_week"].isin([5, 6]).astype(int)
```

Full preprocessing example

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df = pd.DataFrame({
    "Age": [25, None, 40, 35, 50],
    "Salary": [30000, 50000, None, 70000, 90000],
    "City": ["Delhi", "Mumbai", "Delhi", "Chennai", "Mumbai"],
    "Purchased": ["No", "Yes", "Yes", "No", "Yes"]
})

X = df.drop("Purchased", axis=1)
y = df["Purchased"]

numeric_features = ["Age", "Salary"]
categorical_features = ["City"]

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

8. Mini Project Workflow

Use this workflow to practice Phase 2.

Step-by-step project

1. Choose a dataset.
2. Define the target column.
3. Split into train and test data.
4. Build preprocessing with Pipeline and ColumnTransformer.
5. Train baseline models.
6. Evaluate with correct metrics.
7. Use cross validation.
8. Tune hyperparameters with GridSearchCV or RandomizedSearchCV.
9. Compare models.
10. Evaluate the final model once on the test set.

Example models to compare

```
Logistic Regression
Random Forest
XGBoost
SVM
Naive Bayes
```

Practice task

Use the breast cancer dataset and compare:

```
RandomForestClassifier
XGBClassifier
SVC
GaussianNB
```

Print:

```
Accuracy
Precision
Recall
F1-score
Confusion matrix
```

Then tune one model with GridSearchCV.

Final memory points

```
XGBoost: powerful boosting model for tabular data.
Cross validation: more reliable evaluation.
Metrics: choose metric based on the problem.
GridSearchCV: searches best hyperparameters using CV.
SVM: finds maximum-margin boundary.
Naive Bayes: probability-based classifier, great for text.
Feature engineering: improves input data quality.
```